

Name: _____

CSC/MAT 427/627 Programming Project

Chapter 4 – Interpolation

The goal of this project is to implement the various interpolation methods introduced in chapter 4 for approximating a function $f(x)$ over an entire interval $[a, b]$ with a “nicer” function $p(x)$ using only a finite set of values that describe the function f .

First write functions that implement the four methods specified in Section 5 of this project. Pay close attention to the *Programming Hints* in the textbook. Each function will input a set of nodes X and a discrete domain D , and each function will output the y -values corresponding to $p(z)$ for each $z \in D$. Thus, we are approximating the function f over the discrete domain D using the interpolation function p , where p is defined using the nodes in X . Errors will be approximated by finding $\max_{z \in D} |p(z) - f(z)|$. We will also plot the various functions as well as the errors $p - f$.

1 Experiment: Interpolating a nice function

Let $f(x) = \sin(x)$ over the interval $[a, b] = [-3, 3]$ and the fine mesh D be defined by $D = \{z_i\}_{i=0}^{1000}$ for $z_i = a + i \frac{b-a}{1000}$. For each $N = 2, 4, 8, 16, 32, 64$, form the following interpolation functions and calculate the errors over the fine mesh. Record your results in the given table.

1. Lagrange interpolation with uniformly spaced nodes $x_i = a + i \frac{b-a}{N}$ for $i = 0, 1, \dots, N$.
2. Lagrange interpolation with the Chebyshev nodes $x_i = \frac{a+b}{2} + \frac{b-a}{2} \cos\left(\frac{2i+1}{2N+2}\pi\right)$ for $i = 0, 1, \dots, N$.
3. Hermite interpolation with uniformly spaced nodes $x_i = a + i \frac{b-a}{N}$ for $i = 0, 1, \dots, N$.
4. Piecewise quadratic Lagrange interpolation with uniformly spaced nodes $x_i = a + i \frac{b-a}{N}$ for $i = 0, 1, \dots, N$.
5. Cubic complete B-spline interpolation with uniformly spaced nodes $x_i = a + i \frac{b-a}{N}$ for $i = 0, 1, \dots, N$.

	Error = $\max_{z \in D} p(z) - f(z) $				
N	Lagrange 1	Lagrange 2	Hermite	Piecewise Quadratic	Cubic Splines
2					
4					
8					
16					
32					
64					

2 Experiment: Interpolating a non-smooth function

Let $f(x) = \sqrt[3]{x^2}$ over the interval $[a, b] = [-1, 1]$ and the fine mesh D be defined by $D = \{z_i\}_{i=0}^{1000}$ for $z_i = a + i \frac{b-a}{1000}$. For each $N = 6, 12, 24, 48$, form the following interpolation functions and calculate the errors over the fine mesh. Record your results in the given table.

1. Lagrange interpolation with uniformly spaced nodes $x_i = a + i \frac{b-a}{N}$ for $i = 0, 1, \dots, N$.
2. Piecewise cubic Lagrange interpolation with uniformly spaced nodes $x_i = a + i \frac{b-a}{N}$ for $i = 0, 1, \dots, N$.
3. Cubic complete B-spline interpolation with uniformly spaced nodes $x_i = a + i \frac{b-a}{N}$ for $i = 0, 1, \dots, N$.

	Error = $\max_{z \in D} p(z) - f(z) $		
N	Lagrange	Piecewise Cubic	Cubic Splines
6			
12			
24			
48			

Plot all three approximations as well as the original function on the same axes for $N = 6$. Plot all three error functions $p - f$ on the same axes for $N = 12$. Comment on your observations.

3 Experiment: Runge Phenomena

We consider the example in Section 4.12.2 based on the function $f(x) = \frac{1}{1+a^2x^2}$ over the interval $[-b, b]$. Then, the polynomial p_n defined using Lagrange interpolation can be shown to converge for equally spaced nodes if $1 > Cab$ for $C = 0.5255\dots$. Otherwise, the interpolation function p_n will develop large errors between the interpolation points near the endpoints of the domain.

Let $f(x) = \frac{1}{1+2x^2}$ on $[-b, b]$ and the fine mesh D be defined by $D = \{z_i\}_{i=0}^{1000}$ for $z_i = -b + i \frac{2b}{1000}$. Calculate the following with both $b = 1$ and then $b = 4$, and record your results in the corresponding tables.

1. Lagrange interpolation with uniformly spaced nodes $x_i = a + i \frac{b-a}{N}$ for $i = 0, 1, \dots, N$.
2. Lagrange interpolation with the Chebyshev nodes $x_i = \frac{a+b}{2} + \frac{b-a}{2} \cos\left(\frac{2i+1}{2N+2}\pi\right)$ for $i = 0, 1, \dots, N$.
3. Piecewise cubic Lagrange interpolation with uniformly spaced nodes $x_i = a + i \frac{b-a}{N}$ for $i = 0, 1, \dots, N$.
4. Cubic complete B-spline interpolation with uniformly spaced nodes $x_i = a + i \frac{b-a}{N}$ for $i = 0, 1, \dots, N$.

Plot all four approximations as well as the original function on the same axes for $N = 6$. Plot all four error functions $p - f$ on the same axes for $N = 12$. Comment on your observations.

	Error = $\max_{z \in D} p(z) - f(z) $			
N	Lagrange 1	Lagrange 2	Piecewise Cubic	Cubic Splines
6				
12				
24				
48				

$$b = 1$$

	Error = $\max_{z \in D} p(z) - f(z) $			
N	Lagrange 1	Lagrange 2	Piecewise Cubic	Cubic Splines
6				
12				
24				
48				

$$b = 4$$

4 (Grad Students) Application – Collocation Methods for Approximating Boundary Value Problems

The background material for this section of the project can be found in Section 4.9 of the textbook. Consider the boundary value problem (BVP)

$$\begin{cases} -u'' + u = f(x), & 0 < x < 1, \\ u(0) = u(1) = 0. \end{cases}$$

Divide the interval $[0, 1]$ into N equal subintervals $[x_{k-1}, x_k]$, where

$$x_k = kh$$

for $h = \frac{1}{N}$, $k = 0, 1, \dots, N$. We approximate the function $u(x)$ by a cubic spline function $u_h(x)$ defined by

$$u_h(x) = \sum_{k=-1}^{N+1} c_k B_k(x),$$

where $B_k(x)$ denotes a local B-spline basis function centered at x_k . Then, $u_h \in C^2([0, 1])$, and, plugging $u_h(x)$ into the differential equation and accounting for the boundary conditions, we can define a linear system of equations that uniquely determines the constants c_k . Note that this is possible due to the smoothness of the “interpolating” function u_h . The exact form of the linear system is described in Section 4.9, where it is shown that the system is tridiagonal and diagonally dominant. Thus, we can use the tridiagonal solver from Section 2.6 to solve for the coefficients c_k . Numerical methods that approximate boundary value problems by choosing smooth basis functions based on a set of points x_k and using a linear combination/expansion to form an approximation that

can be directly inserted into the differential equation to form a set of equations for the unknown coefficients are referred to as collocation methods. This differs from the finite difference method considered in Section 2.7 where we formed an approximation for u only at the discrete points x_k since u_h is defined over the entire interval. Instead of approximating the derivatives using difference quotients, we approximate the function itself using a finite expansion and find the exact solution (using exact derivatives) when restricting our search to functions in the finite space generated by the smooth basis functions.

Let $f(x) = 4e^{-x} - 4xe^{-x}$ so that the exact solution is $u(x) = x(1-x)e^{-x}$. Approximate $u(x)$ for $N = 5, 10, 20, 40, 80, 160$ and fill out the following table, where the fine mesh D is defined by $D = \{z_i\}_{i=0}^{1000}$ for $z_i = i \frac{1}{1000}$:

h	Error: $E_h = \max_{z \in D} u_h(z) - u(z) $	Rate = $\ln(E_{2h}/E_h)/\ln(2)$
0.2		N/A
0.1		
0.05		
0.025		
0.0125		
0.00625		

Plot the exact solution and the approximation on the same axes using the points from the fine grid for $N = 5$. Plot the error $u_h(x) - u(x)$ using the points from the fine grid for $N = 20$. Comment on your observations.

5 Interpolation Methods

In all of the methods we assume the function f is defined externally. The function could also be input into the method if allowed by the programming language you choose. Do not hard-code the function f inside a method.

For the external function f :

Input: x, p (p is an integer).

Output: $f^{(p)}(x)$.

For $p = 0$, we have the output is $f(x)$. You only need to code the derivatives for $p = 0, 1$.

Implement the following methods as separate functions. All of the methods assume $x_0 \geq a$ and $x_N \leq b$, with equality when using uniformly spaced nodes. This assumption ensures all of the methods are using the same number of nodes with the same indexing.

1. Lagrange Interpolation using Divided Differences

Your implementation should combine Algorithms 4.1 and 4.2 in Section 4.2 with the needed changes to match the specified inputs and outputs for the project.

Input: $X = \{x_i\}_{i=0}^N$, $D = \{z_i\}_{i=0}^{1000}$, $(f, f(X))$, and/or N if needed

Output: $p_N(D) = \{p_N(z_i)\}_{i=0}^{1000}$

2. Hermite Interpolation using Divided Differences

The code will define outputs for a function H_N such that

$$H_N(x_i) = f(x_i), \quad H'_N(x_i) = f'(x_i)$$

for all $i = 0, 1, \dots, N$. Note that H_N is a polynomial with degree $\leq 2N + 1$.

Input: Input: $X = \{x_i\}_{i=0}^N$, $D = \{z_i\}_{i=0}^{1000}$, $(f, f', f(X), f'(X))$, and/or N if needed)

Output: $H_N(D) = \{H_N(z_i)\}_{i=0}^{1000}$

Algorithm (Based on modifying Algorithms 4.1 and 4.2)

We let H_N be the Hermite polynomial such that

$$\begin{aligned} H_N(x) = & Q_{0,0} + Q_{1,1}(x - x_0) + Q_{2,2}(x - x_0)^2 + Q_{3,3}(x - x_0)^2(x - x_1) \\ & + Q_{4,4}(x - x_0)^2(x - x_1)^2 + \dots \\ & + Q_{2N+1,2N+1}(x - x_0)^2(x - x_1)^2 \dots (x - x_N), \end{aligned}$$

where $Q_{i,i}$ will denote the divided difference coefficients.

```
! Define  $Q_{i,i}$  for  $i = 0, \dots, 2N + 1$ 
for i=0 to N do
    xx(2i) = x(i)
    xx(2i+1) = x(i)
    Q(2i,0) = f(x(i))
    Q(2i+1,0) = f(x(i))
    Q(2i+1,1) = f'(x(i))
    if (i  $\neq$  0)
        Q(2i,1) = (Q(2i,0) - Q(2i-1,0)) / (xx(2i) - xx(2i-1))
    end if
end for
for i=2 to 2N+1 do
    for j=2 to i do
        Q(i,j) = (Q(i,j-1) - Q(i-1,j-1)) / (xx(i) - xx(i-j))
    end for
end for

! Define  $H_N(z_k)$ 
for k=0 to 1000 do
    HN(k) = Q(2N+1,2N+1) * (z(k) - x(N))
    for i=N-1 down to 0
        HN(k) = (z(k) - x(i)) * (Q(2i+1,2i+1) + Q(2i+2, 2i+2)*(z(k) - x(i)) +
            + HN(k)*(z(k) - x(i)))
    end for
    HN(k) = HN(k) + Q(0,0)
end for
```

3. Piecewise Lagrange Interpolation

Your implementation should be based on Algorithm 4.3 in Section 4.7 with the needed changes to match the specified inputs and outputs for the project. Your input will specify a value d that represents the degree of the local polynomials used to build the piecewise polynomial p_N .

Input: $X = \{x_i\}_{i=0}^N$, $D = \{z_i\}_{i=0}^{1000}$, d , $(f, f(X))$, and/or N if needed

Output: $p_N(D) = \{p_N(z_i)\}_{i=0}^{1000}$

When implementing Algorithm 4.3 you will need to build a search function that determines which subinterval you are on so that you can choose the correct piece of the piecewise polynomial when evaluating $p_N(x)$ for a given value x . Note that this can be done using a floor function as mentioned on page 207 since we will always be using equally spaced nodes in the project when using piecewise Lagrange interpolation.

4. Cubic complete B-splines

Input: $X = \{x_i\}_{i=0}^N$, $D = \{z_i\}_{i=0}^{1000}$, $(f, f(X))$, and/or N if needed

Output: $q_N(D) = \{q_N(z_i)\}_{i=0}^{1000}$

Let $h = \frac{b-a}{N}$, and define the nodes $x_i = a + ih$ for $i = -3, -2, \dots, N+3$ so that

$$\begin{aligned} x_{-3} = a - 3h < x_{-2} < x_{-1} < x_0 = a < x_1 < \dots \\ < x_N = b < x_{N+1} < x_{N+2} < x_{N+3} = a + (N+3)h. \end{aligned}$$

Your code should construct the spline interpolation function $q_N(x)$ such that

$$q_N(x) = \sum_{i=-1}^{N+1} c_i B_i(x) \quad (1)$$

for $B_i(x)$ local basis functions defined such that

$$B_i(x) = B((x - x_i)/h)$$

for the parent function B given by

$$B(x) = \begin{cases} 0, & x \leq -2, \\ (x+2)^3, & -2 < x \leq -1, \\ 1 + 3(x+1) + 3(x+1)^2 - 3(x+1)^3, & -1 < x \leq 0, \\ 1 + 3(1-x) + 3(1-x)^2 - 3(1-x)^3, & 0 < x \leq 1, \\ (2-x)^3, & 1 < x \leq 2, \\ 0, & x > 2. \end{cases} \quad (2)$$

Using the tridiagonal system of equations solver from Section 2.6, you can solve the equation

$$\begin{bmatrix} 4 & 2 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 0 & 1 & 4 & 1 \\ & \ddots & \ddots & \ddots \\ & & 1 & 4 & 1 & 0 \\ & & 0 & 1 & 4 & 1 \\ & & 0 & 0 & 2 & 4 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ \vdots \\ c_{N-2} \\ c_{N-1} \\ c_N \end{bmatrix} = \begin{bmatrix} f(x_0) + \frac{1}{3}hf'(x_0) \\ f(x_1) \\ f(x_2) \\ \vdots \\ f(x_{N-2}) \\ f(x_{N-1}) \\ f(x_N) - \frac{1}{3}hf'(x_N) \end{bmatrix} \quad (3)$$

for the coefficients $c_0, c_1, \dots, c_N$. We then let $c_{-1} = c_1 - \frac{1}{3}hf'(x_0)$ and $c_{N+1} = c_{N-1} + \frac{1}{3}hf'(x_N)$. Once the coefficients are known, we can evaluate (1) for any value x .

To implement the spline interpolation method you need to

- (a) Define the function $B(x)$ that inputs a value x and outputs a value $y = B(x)$ using (2).
- (b) Define the vectors/arrays ℓ , d , and u corresponding to the subdiagonal, diagonal, and super-diagonal entries in the matrix used to define (3) as well as a vector b corresponding to the right-hand-side of (3).
- (c) Solve the system of equations using the code from Section 2.6 for the coefficients c_i . Also define c_{-1} and c_{N+1} using the formula above corresponding to complete splines.
- (d) Evaluate (1) for each point $z_k \in D$ using the following code:

```
for k=0 to 1000
    sum = 0
    for i = -1 to N+1
        sum = sum + c(i) * B( (z(k) - x(i)) / h )
    end for
    qN(k) = sum
end for
```

Note that you can also drastically reduce the range of i to only four integers if you first determine an index j such that $z_k \in [x_{j-1}, x_j]$.